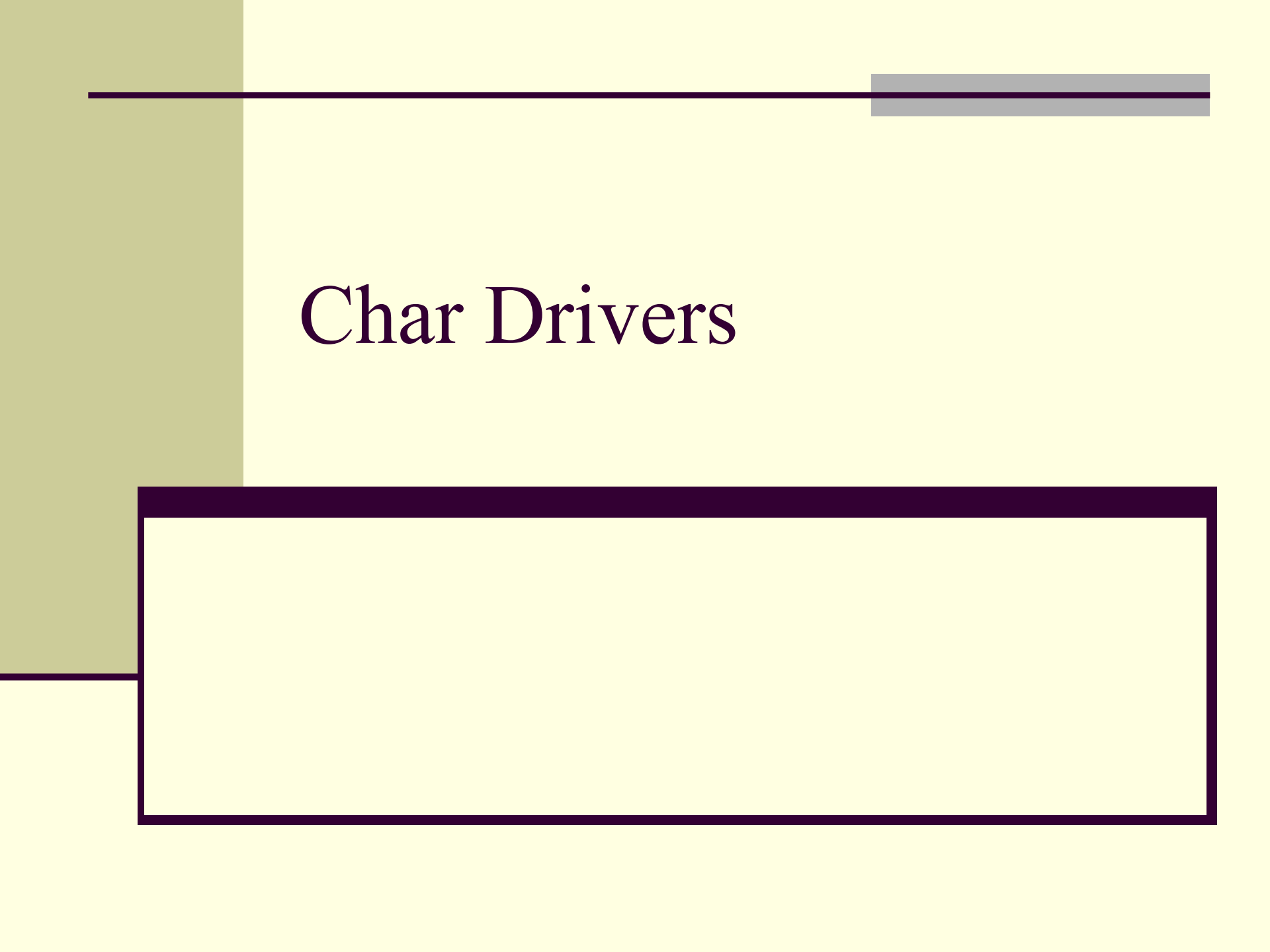




Char Drivers

SCULL: PSEUDO-DEVICE

Example char-type device driver

Introduction

- A complete char device driver
- **scull**
 - **S**imple **C**haracter **U**tility for **L**oading **L**ocalities
 - Kernel allocated memory treated as device
 - Not hardware dependent
 - Explore interface between char driver and kernel
- The *scull* source implements the following devices. Each kind of device implemented by the module is referred to as a *type*.

The Design of **scull**

- Implements various devices
- **scull0** to **scull3**
 - Four device drivers, each consisting of a memory area
 - Global
 - Data contained within the device is shared by all the file descriptors that opened it
 - Persistent
 - If the device is closed and reopened, data isn't lost

The Design of **scull**

- **scullpipe0 to scullpipe3**
 - Four FIFO devices
 - Act like pipes
 - Illustrate how blocking and non-blocking read and write can be implemented

The Design of **scull**

- Variants of **scull0**
 - Illustrate typical driver-imposed access limitations
 - **scullsingle**
 - Similar to **scull0**
 - Only one process can use the driver at a time
 - **scullpriv**
 - Private to each virtual console

The Design of **scull**

- One user at a time
- **sculluid**
 - Can be opened multiple times by one user
 - **Fails** on open() if another user is locking the device
 - Returns “Device Busy”
- **scullwuid**
 - **Blocks** on open() if another user is locking the device

MAJOR AND MINOR DEVICE NUMBERS

Identifying a Device

Major and Minor Device Numbers

- Char devices are accessed through names in the file system
 - Abstraction for handling devices
 - Special files in `/dev`
 - Implemented using inode data structure

```
> cd /dev
```

```
> ls -l
```

```
crw----- 1 root  root      5,      1 Apr 12 16:50 console
brw-rw---- 1 root  disk      8,      0 Apr 12 16:50 sda
brw-rw---- 1 root  disk      8,      1 Apr 12 16:50 sda1
```

Major and Minor Device Numbers

- Char devices are accessed through names in the file system
 - Abstraction for handling devices
 - Special files in **/dev**
 - Implemented using inodes

```
> cd /dev
```

```
> ls -l
```

```
crw----- 1 root  root
```

```
brw-rw---- 1 root  disk
```

```
brw-rw---- 1 root  disk
```

Block drivers
are identified
by a "b"

Char drivers
are identified
by a "c"

Major numbers

5,	1	Apr 12 16:50	console
8,	0	Apr 12 16:50	sda
8,	1	Apr 12 16:50	sda1

Minor numbers

Major and Minor Device Numbers

- Major number ***traditionally*** identifies the **device driver**
 - Class of device (traditionally)
 - E.g.,
 - `/dev/sda` and `/dev/sda1` are managed by driver 8
 - `cat /proc/devices`
 - Map number to name of device driver
 - Can have more than one major to a single driver but not typical
- Minor number specifies the particular **device**

The Internal Representation of Device Numbers

- **dev_t** type, defined in `<linux/types.h>`
- Macros defined in `<linux/kdev_t.h>`
 - 12 bits for the major number
 - Use **MAJOR(dev_t dev)** to obtain the major number
 - 20 bits for the minor number
 - Use **MINOR(dev_t dev)** to obtain the minor number
- Use **MKDEV(int major, int minor)** to turn them into a **dev_t**

Allocating and Freeing Device Numbers

- Register a major device number (old way)

```
int register_chrdev_region(dev_t first,  
    unsigned int count, char *name);
```

- **first**

- Beginning device number
- Minor device number is often 0

- **count**

- Requested number of contiguous device numbers

- **name**

- Name of the device

- **return**

- 0 on success, error code on failure

Allocating and Freeing Device Numbers

- Kernel can allocate a major number on the fly (dynamic major number)

```
int alloc_chrdev_region(dev_t *dev,  
    unsigned int firstminor, unsigned int  
    count, char *name);
```

- **dev**

- Output-only parameter that holds the first number on success

- **firstminor**

- Requested first minor number
- Often 0

Allocating and Freeing Device Numbers

- To free your device numbers, use

```
int unregister_chrdev_region(dev_t first,  
    unsigned int count);
```

Allocating and Freeing Device Numbers

- Some major device numbers are statically assigned
 - See `Documentation/devices.txt`
- To avoid conflicts, use dynamic allocation
- Creates `/proc/devices` entries, but does not create the device nodes in the filesystem

scull_load Shell Script

```
#!/bin/sh
module="scull"
device="scull"
mode="664"

# invoke insmod with all arguments we got and use a pathname,
# as newer modutils don't look in . by default
/sbin/insmod ./ $module.ko $* || exit 1

# remove stale nodes
rm -f /dev/${device}[0-3]

major=$(awk "\$2==\"$module\" {print \$1}" /proc/devices)
```

scull_load Shell Script

```
mknod /dev/${device}0 c $major 0
mknod /dev/${device}1 c $major 1
mknod /dev/${device}2 c $major 2
mknod /dev/${device}3 c $major 3
```

```
# give appropriate group/permissions, and change the group.
# Not all distributions have staff, some have "wheel" instead.
```

```
group="staff"
```

```
grep -q `^staff:` /etc/group || group="wheel"
```

```
chgrp $group /dev/${device}[0-3]
```

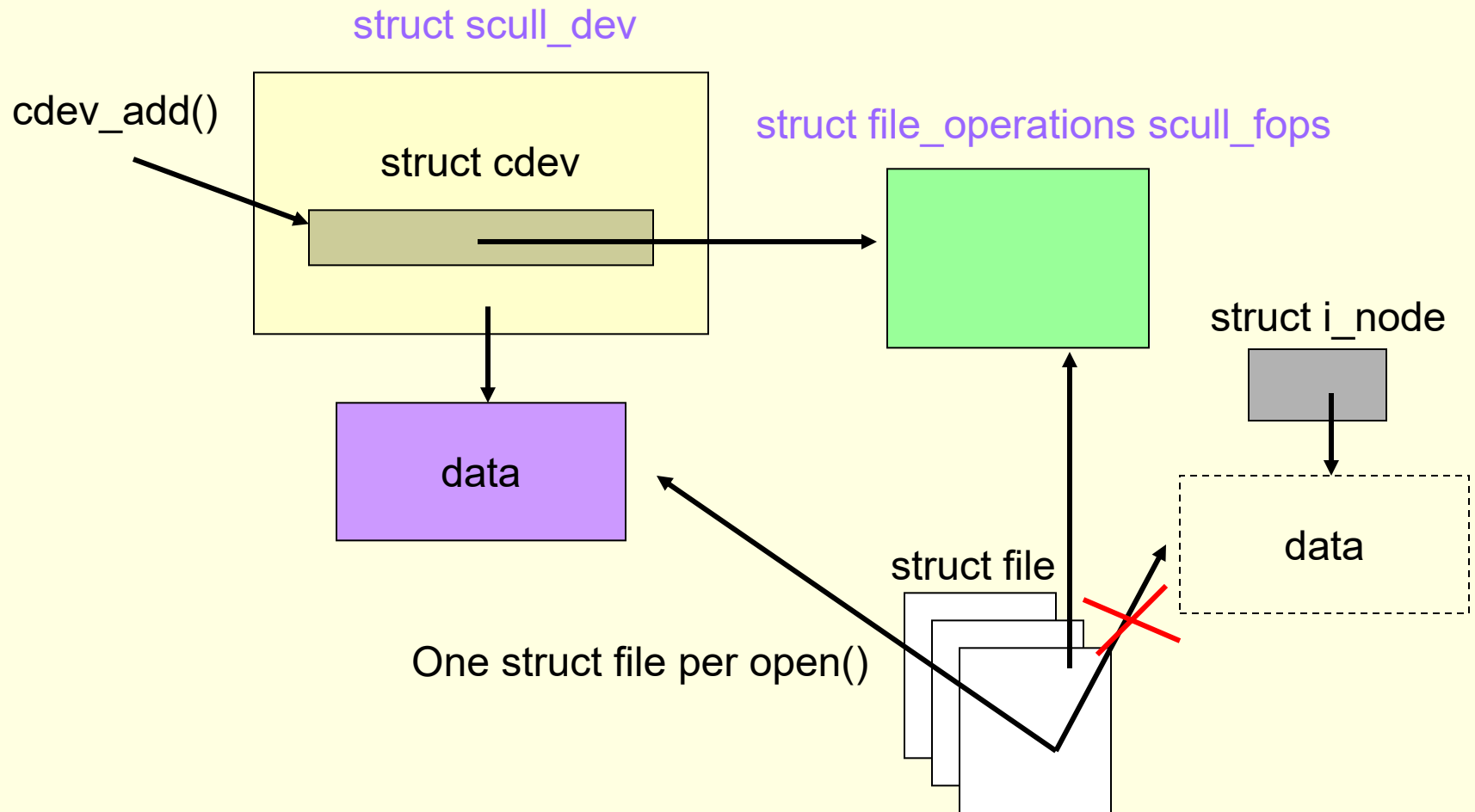
```
chmod $mode /dev/${device}[0-3]
```



CHAR DEVICE DATA STRUCTURES



Overview of Data Structures



Some Important Data Structures

- `file_operations`
- `file`
- `inode`

- Defined in `<linux/fs.h>`

File Operations

```
struct file_operations {  
    /* pointer to the module that owns the structure prevents  
       the module from being unloaded while in use */  
    struct module *owner;  
  
    /* change the current position in a file  
       returns a 64-bit offset, or a negative value on errors  
    */  
    loff_t (*llseek) (struct file *, loff_t, int);  
  
    /* returns the number of bytes read, or a negative value  
       on errors */  
    ssize_t (*read) (struct file *, char __user *, size_t,  
                     loff_t *);
```

File Operations

```
/* returns the number of written bytes, or a negative
   value on error */
```

```
ssize_t (*write) (struct file *, const char __user *,
                  size_t, loff_t *);
```

```
/* first operation performed on the device file
   if not defined, opening always succeeds, but driver
   is not notified */
```

```
int (*open) (struct inode *, struct file *);
```

```
/* invoked when the file structure is being released */
```

```
int (*release) (struct inode *, struct file *);
```

```
/* provides a way to issue device-specific commands
   (e.g., formatting) */
```

```
int (*unlocked_ioctl) (struct file *, unsigned int,
                       unsigned long);
```

```
int (*compat_ioctl) (struct file *, unsigned int,
                     unsigned long);
```

... many more of struct file_operations members not covered in this lecture

scull Device Driver

- Implements only a few of the methods:

```
struct file_operations scull_fops = {  
    .owner = THIS_MODULE,  
    .llseek = scull_llseek,  
    .read = scull_read,  
    .write = scull_write,  
    .unlocked_ioctl = scull_ioctl,  
    .open = scull_open,  
    .release = scull_release,  
};
```


The **file** Structure

- Describes an open file
- What Unix calls an open **file descriptor**
- Allocated when a file/device is opened
- **ref count** incremented when new references are created, e.g. by dup and fork
- freed on "last close" of a file/device
- Contains a reference to a file_operations structure

The **file** Structure

- A pointer to file is often called **filp**
- Some important fields
 - **fmode_t f_mode;**
 - File properties (set by kernel based on open() parameters)
 - E.g., readable (**FMODE_READ**) or writable (**FMODE_WRITE**)
 - **loff_t f_pos;**
 - Current reading/writing position (64-bits)
 - **unsigned int f_flags;**
 - File flags (combined flags/mode from open())
 - E.g., **O_RDONLY**, **O_NONBLOCK**, **O_SYNC**

The File Structure

- Some important fields

- **struct file_operations *f_op;**

- Operations associated with the file

- Dynamically replaceable pointer

- Equivalent of method overriding in OO programming

- **void *private_data;**

- Can be used to store additional data structures

- Needs to be freed during the **release** method

The File Structure

- Some important fields

- `struct dentry *f_dentry;`

- Directory entry associated with the file

- Used to access the inode data structure

- `filp->f_dentry->d_inode`

The i-node Structure

- There can be numerous file structures (multiple open descriptors) for a single file
- Only one inode structure per file

The i-node Structure

- Some important fields

- `dev_t i_rdev;`

- Contains device number

- For portability, use the following macros

- `unsigned int iminor(struct inode *inode);`

- `unsigned int imajor(struct inode *inode);`

- `struct cdev *i_cdev;`

- Contains a pointer to the data structure that refers to a char device file

CHAR DEVICE REGISTRATION

Char Device Registration

- **struct cdev** to represent char devices

```
#include <linux/cdev.h>
```

```
/* first way - allocates and initializes cdev */
```

```
struct cdev *my_cdev = cdev_alloc();
```

```
my_cdev->ops = &my_fops;
```

```
/* second way - initialize already allocated cdev  
(see scull driver) */
```

```
void cdev_init(struct cdev *cdev, struct  
file_operations *fops);
```


Char Device Registration

- Either way
 - Need to initialize `file_operations` and set `owner` to `THIS_MODULE`
- Inform the kernel by calling

```
int cdev_add(struct cdev *dev, dev_t
num, unsigned int count);
```

 - `num`: first device number
 - `count`: number of device numbers
- Remove a char device, call this function

```
void cdev_del(struct cdev *dev);
```

Allocating and Freeing Device Numbers

- **register_chrdev()**
 - **Consolidates into one call functionality of:**
 - `alloc_chrdev_region()`
 - `cdev_add()`
 - **Allocates 256 minor devices**
 - **Unregister counterpart**
 - `unregister_chrdev()`

Device Registration in **scull**

- **scull** represents each device with **struct scull_dev**

```
struct scull_dev {  
    struct scull_qset *data; /* pointer to first quantum set */  
    int quantum;             /* the current quantum size */  
    int qset;                /* the current array size */  
    unsigned long size;      /* amount of data stored here */  
    unsigned int access_key; /* used by sculluid & scullpriv */  
    struct mutex mutex;      /* mutual exclusion */  
    struct cdev cdev;        /* char device structure */  
};
```

Char Device Initialization Steps

- Register device driver name and numbers
- Allocation of the `struct scull_dev` objects
- Initialization of scull cdev objects
 - Calls `cdev_init` to initialize the `struct cdev` component
 - Sets `cdev.owner` to this module
 - Sets `cdev.ops` to `scull_fops`
 - Calls `cdev_add` to complete registration

Char Device Cleanup Steps

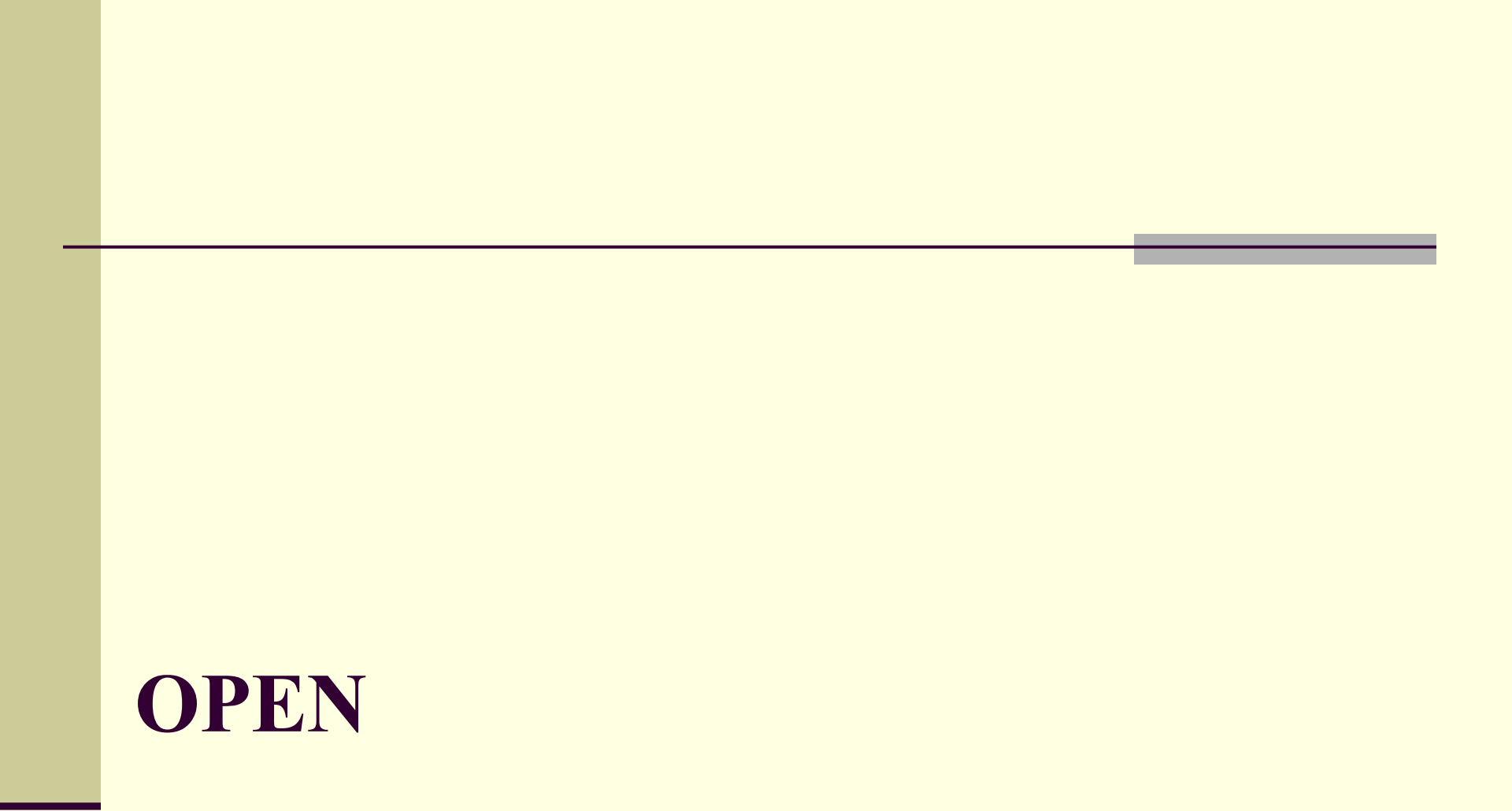
- Clean up internal data structures
- `cdev_del scull` devices
- Deallocate `scull` devices
- Unregister device numbers

Device Registration in **scull**

- To add **struct scull_dev** to the kernel

```
static void scull_setup_cdev(struct scull_dev *dev, int index)
{
    int err, devno = MKDEV(scull_major, scull_minor + index);

    cdev_init(&dev->cdev, &scull_fops);
    dev->cdev.owner = THIS_MODULE;
    err = cdev_add(&dev->cdev, devno, 1);
    if (err) {
        printk(KERN_NOTICE "Error %d adding scull%d", err,
                index);
    }
}
```



OPEN

The **open** Method

- In most drivers, **open** should
 - Check for device-specific errors
 - Initialize the device (if opened for the first time)
 - Update the `f_op` pointer, as needed
 - Set pointer to locate needed data in subsequent function calls (e.g., `read`, `write`)
`filp->private_data`
 - Check flags
 - `O_NONBLOCK` flag
 - Generally ignored by filesystems
 - Return immediately if open would block
 - By default open may block until file is ready
 - `if (filp->f_flags & O_NONBLOCK) return -EAGAIN;`
`access.c`

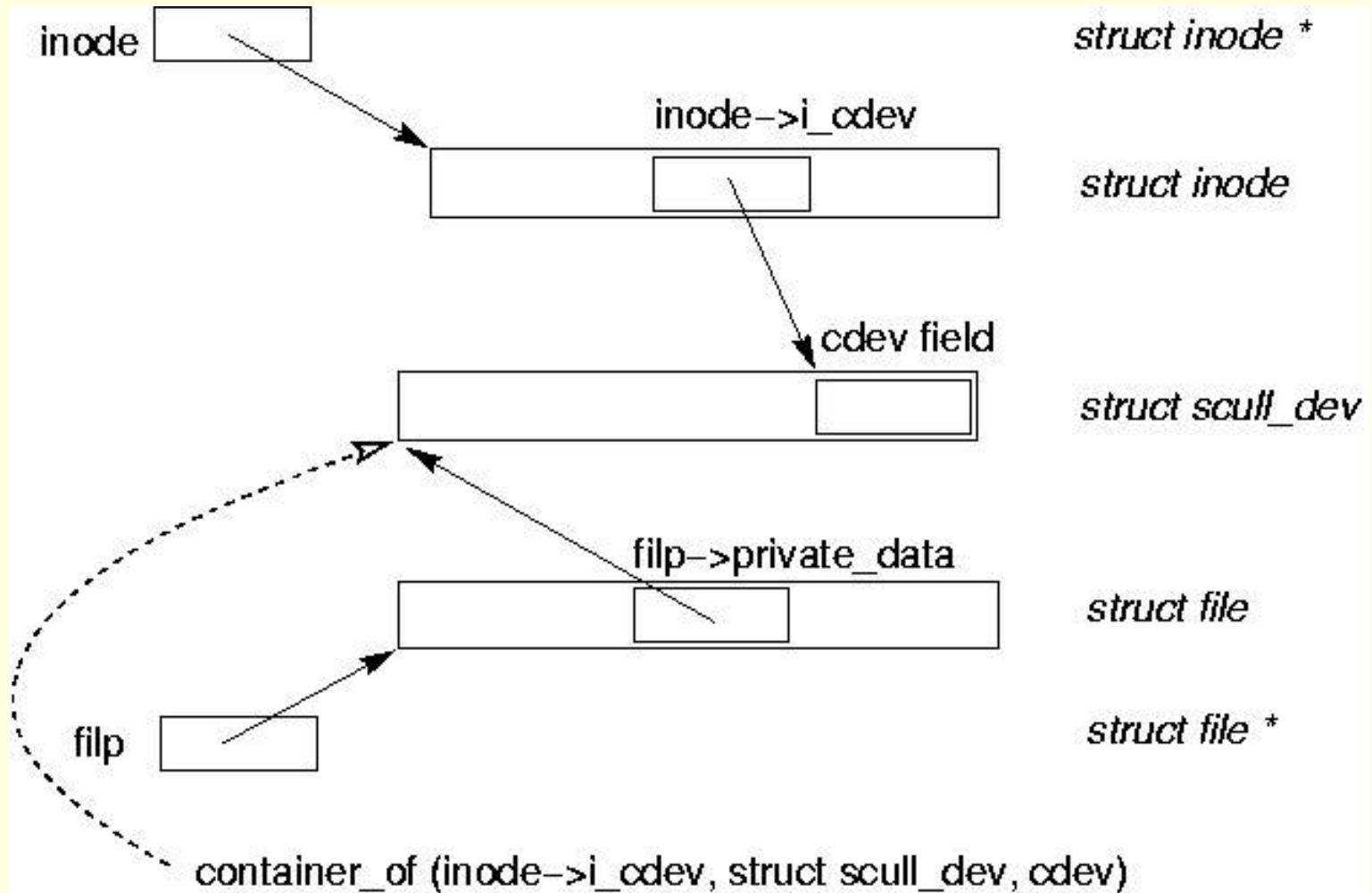
The **open** Method

```
int scull_open(struct inode *inode, struct file *filp) {
    struct scull_dev *dev; /* device info */

    /* #include <linux/kernel.h>
       container_of(pointer, container_type, container_field
       returns the starting address of struct scull_dev */
    dev = container_of(inode->i_cdev, struct scull_dev, cdev);
    filp->private_data = dev;

    /* now trim to 0 the length of the device if open was
       write-only */
    if ((filp->f_flags & O_ACCMODE) == O_WRONLY) {
        scull_trim(dev); /* ignore errors */
    }
    return 0; /* success */
}
```

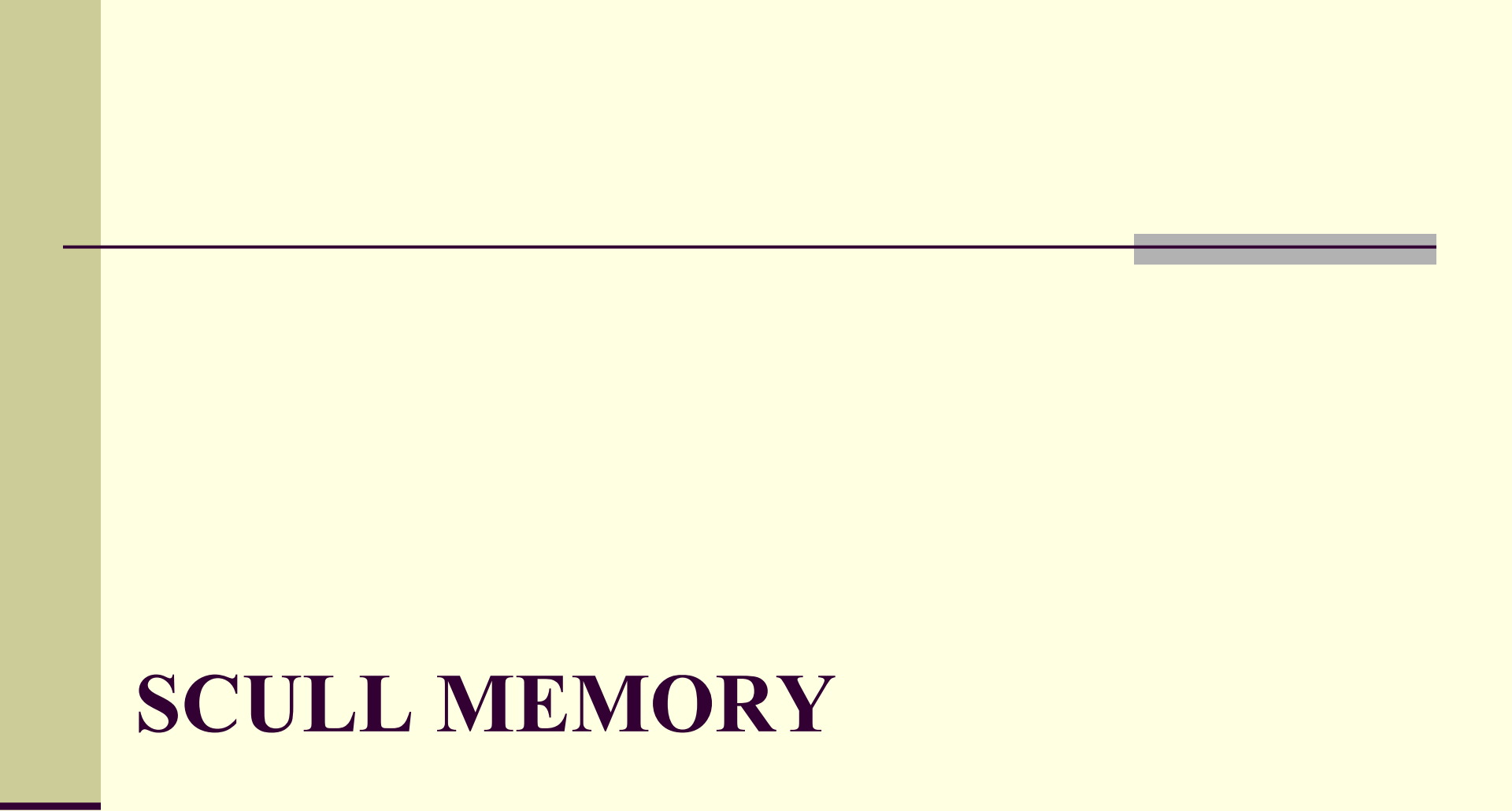
The **open** Method



The **release** Method

- Deallocate `filp->private_data`
- Shut down the device on last close
 - One **release** call per **open**
 - Potentially multiple **close** calls per **open** due to **fork/dup**
- **scull** has no hardware to shut down

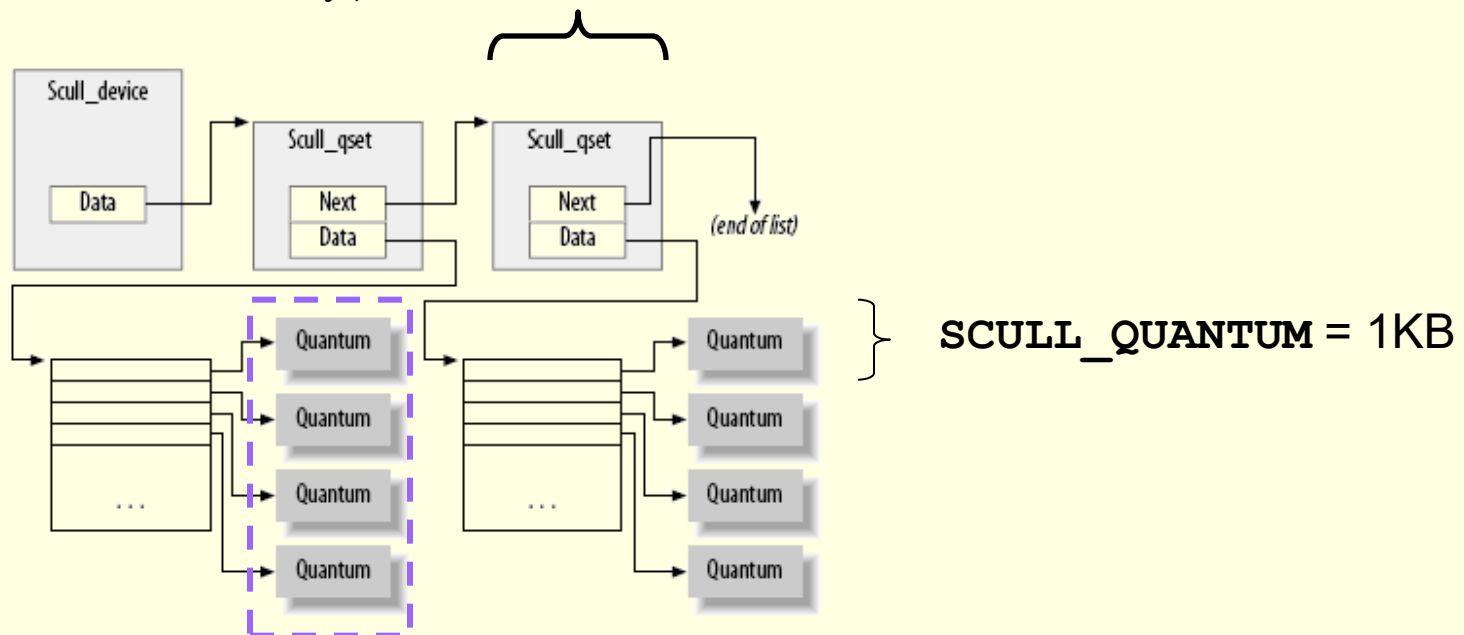
```
int scull_release(struct inode *inode, struct file *filp) {  
    return 0;  
}
```



SCULL MEMORY

scull's Memory Usage

```
struct scull_qset {
    void **data;
    struct scull_qset *next;
};
```



Quantum set, **SCULL** **QSET** = 1K quanta

scull's Memory Usage

- Dynamically allocated
- `#include <linux/slab.h>`
 - `void *kmalloc(size_t size, int flags) ;`
 - Allocate **size** bytes of memory
 - For now, always use **GFP_KERNEL**
 - Return a pointer to the allocated memory, or **NULL** if the allocation fails
 - `void kfree(void *ptr) ;`

scull's Memory Usage

```
int scull_trim(struct scull_dev *dev) {
    struct scull_qset *next, *dptr;
    int qset = dev->qset; /* dev is not NULL */
    int i;

    for (dptr = dev->data; dptr; dptr = next) {
        if (dptr->data) {
            for (i = 0; i < qset; i++) kfree(dptr->data[i]);
            kfree(dptr->data);
            dptr->data = NULL;
        }
        next = dptr->next;
        kfree(dptr);
    }
    dev->size = 0; dev->data = NULL;
    dev->quantum = scull_quantum; dev->qset = scull_qset;
    return 0;
}
```

Race Condition Protection

- Different processes may try to execute operations on the same scull device concurrently
- There would be trouble if both were able to access the data of the same device at once
- `scull` avoids this using a per-device mutex
- All operations that touch the device's data need to lock the mutex

Race Condition Protection

- Some mutex usage rules
 - No double locking
 - No double unlocking
 - Always lock at start of critical section
 - Don't release until end of critical section
 - Don't forget to release before exiting
 - return, break, or goto
 - If you need to hold two locks at once, lock them in a well-known order, unlock them in the reverse order (e.g., lock1, lock2, unlock2, unlock1)

Mutex Usage Examples

■ Initialization

```
mutex_init(&scull_devices[i].mutex);
```

■ Critical section

```
if (mutex_lock_interruptible(&dev->mutex))  
    return -ERESTARTSYS;  
scull_trim(dev); /* ignore errors */  
mutex_unlock(&dev->mutex);
```

Mutex vs. Spinlock

- Mutex may block
 - Calling process is blocked until the lock is released
- Spinlock may spin (loop)
 - Calling processor spins until the lock is released
- Never call “lock” unless it is OK for the current thread to block
 - Do not call “lock” while holding a spinlock
 - Do not call “lock” within an interrupt handler



READ AND WRITE

read and write

- Kernel memory is locked into real memory so it is always resident
- User memory may have pages that are not resident
- If kernel attempts to access user pages there may be a page fault
 - Causes the faulting process to be blocked until the page is fetched

read and write

```
ssize_t (*read) (struct file *filp, char __user *buff,  
                size_t count, loff_t *offp);  
ssize_t (*write) (struct file *filp, const char __user *buff,  
                 size_t count, loff_t *offp);
```

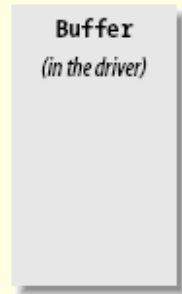
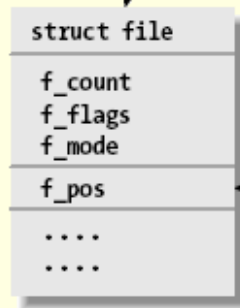
- **filp**: file pointer
- **buff**: a user-space pointer
 - May not be valid in kernel mode
 - Could be malicious
 - Might be swapped out
- **count**: size of requested transfer
- **offp**: file position pointer

read and write

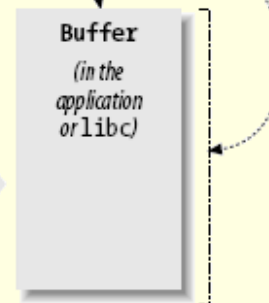
- To safely access user-space buffer - cannot be directly dereferenced by kernel code.
 - Use kernel-provided functions
 - `#include <linux/uaccess.h>`
 - `unsigned long copy_to_user(void __user *to, const void *from, unsigned long count);`
 - `unsigned long copy_from_user(void *to, const void __user *from, unsigned long count);`
 - Check whether the user-space pointer is valid
 - Return the amount of memory still to be copied

read and write

```
ssize_t dev_read(struct file *file, char *buf, size_t count, loff_t *ppos);
```



copy_to_user()



Kernel Space
(nonswappable)

User Space
(swappable)

The **read** Method

- Return values
 - Equals to the **count** argument, we are done
 - Positive < **count**, retry
 - 0, end-of-file
 - Negative, check **<linux/errno.h>**
 - Common errors
 - **-EINTR** (interrupted system call)
 - **-EFAULT** (bad address)
- May block
 - No data, but will arrive later
 - **if (filp->f_flags & O_NONBLOCK)** [\[pipe.c\]](#)
- May be interrupted (e.g., signal)

The **read** Method

- Each **scull_read** deals only with a single data quantum
 - I/O library may reiterate the call to read additional data
 - If read position > device size, return 0 (end-of-file)

The **read** Method

```
ssize_t scull_read(struct file *filp, char __user *buf, size_t count,
                  loff_t *f_pos)
{
    struct scull_dev *dev = filp->private_data;
    struct scull_qset *dptr;      /* the first listitem */
    int quantum = dev->quantum, qset = dev->qset;
    int itemsize = quantum * qset; /* how many bytes in the listitem */
    int item, s_pos, q_pos, rest;
    ssize_t retval = 0;

    if (mutex_lock_interruptible(&dev->mutex)) // Locking the device, other process
                                                can't able to modify
        return -ERESTARTSYS;
    if (*f_pos >= dev->size)
        goto out;
    if (*f_pos + count > dev->size)
        count = dev->size - *f_pos;
```

The **read** Method

```
/* find listitem, qset index, and offset in the quantum */
item = (long)*f_pos / itemsize;
rest = (long)*f_pos % itemsize;
s_pos = rest / quantum; q_pos = rest % quantum;

/* follow the list up to the right position (defined elsewhere) */
dptr = scull_follow(dev, item);

if (dptr == NULL || !dptr->data || ! dptr->data[s_pos])
    goto out; /* don't fill holes */

/* read only up to the end of this quantum */
if (count > quantum - q_pos)
    count = quantum - q_pos;
```

The **read** Method

```
if (copy_to_user(buf, dptr->data[s_pos] + q_pos, count)) {  
    retval = -EFAULT;  
    goto out;  
}  
*f_pos += count;  
retval = count;  
  
out:  
    mutex_unlock(&dev->mutex);  
    return retval;  
}
```

Playing with the New Devices

- With **open**, **release**, **read**, and **write**, a driver can be compiled and tested
- Use **free** command to see the memory usage of **scull**
- Use **strace** to monitor various system calls and return values
- **strace ls -l > /dev/scull0** to see quantized reads and writes